



GH-600 Practice Exam

Developing in Agentic AI Systems — Practice Mode

10 questions · Friday, May 22, 2026

Correct answers highlighted · Full explanations included

Questions & Answers

Q1**Domain C: Memory, state, and execution**

single choice

hard

A backend team uses Copilot cloud agent over several days to modernize a payment service. On the third run, the agent repeats completed refactors and starts changing logging patterns that conflict with the original issue requirements. The team also wants future Copilot code reviews in the same repository to remember the repository's database-connection convention. Which memory and state strategy should the team implement?

The team needs continuity for the long-running migration, but each new issue must start from a clean task context. Stable repository conventions should be reusable by Copilot features in the same repository.

- A Store task progress, issue decisions, and repository conventions only in Copilot Memory, then resume future runs from remembered facts.
- B Persist checkpoints in the PR, re-anchor each run to the issue criteria, and store only stable repository conventions in Copilot Memory. ✓**
- C Move checkpoints and conventions into a shared external vector store, then make all service repositories read the same memory.
- D Keep the full chat and tool-call history pinned across related issues, pruning it only after the whole migration completes.

✓ **Why the correct answer is right**

B separates durable task state from long-term repository memory. Progress checkpoints in the PR let the agent resume without repeating work, re-anchoring to the issue criteria corrects context drift, and Copilot Memory is appropriate for stable repository-level facts such as coding conventions or architectural decisions that may later be used by cloud agent or code review in the same repository.

✗ **Why the other options are wrong**

- **A:** Copilot Memory is designed for repository facts and user preferences, not as the only durable execution log for a multi-day task.
- **D:** Keeping full historical context increases drift and stale-context risk; completed tool history should be pruned instead of carried across task boundaries.
- **C:** A shared cross-repository memory risks conflicting or stale context; Copilot repository-level facts are scoped to operations on the same repository.

💡 For GH-600 memory questions, separate execution state from reusable knowledge: checkpoints belong in durable GitHub artifacts, while stable repo conventions belong in scoped Copilot Memory.

memory-strategy

state-persistence

context-drift

copilot-memory

Q2**Domain F: Guardrails, safety, accountability, and governance**

single choice

hard

A DevOps team wants Copilot cloud agent to implement routine application fixes, but some fixes may update deployment workflows and trigger a staging deployment. The security team wants to preserve delivery speed while adding checkpoints only where risk is meaningful. What should the team configure?

The repository already requires pull requests for default-branch changes. The team must prevent unreviewed deployment changes without blocking low-risk code edits.

- A Allow workflows to run automatically, then review session logs and rollback if the staging deployment fails.
- B Require security approval for every agent-created commit and prohibit staging deployments until the quarterly access review finishes.
- C Use rulesets with required checks, CODEOWNERS on agent configuration, and environment approval for staging deployments. ✓**
- D Move all deployment logic outside GitHub Actions so Copilot PRs cannot trigger any deployment workflow.

✓ **Why the correct answer is right**

This right-sizes the guardrails: rulesets and required checks apply to Copilot cloud agent like human contributors, CODEOWNERS review protects Copilot and MCP configuration files, and environment approvals create a human checkpoint for deployment risk without slowing routine code changes.

✗ **Why the other options are wrong**

- **B:** Requiring approval for every commit over-controls low-risk work and slows delivery without focusing on deployment or configuration risk.
- **A:** Reviewing logs after automatic workflow execution is reactive and does not provide a pre-deployment human checkpoint.
- **D:** Moving deployment outside GitHub Actions adds avoidable architectural complexity instead of using existing rulesets and environment controls.

💡 For GH-600 guardrail questions, prefer layered preventive controls that match the risk level. Avoid blanket approvals for low-risk work and avoid relying on monitoring after risky actions occur.

autonomy-levels

human-in-the-loop

rulesets

environment-protection

least-privilege

Q3**Domain B: MCP/tool use and environment permissions****code or config artifact****very_hard**

A repository administrator is reviewing an MCP configuration before enabling a Copilot cloud agent for bug triage. The agent needs read-only GitHub context from the current repository and read-only incident details from Sentry. What remediation should the administrator apply before enabling the agent?

Security requires that the agent avoid broad GitHub access, use Copilot cloud agent secret substitution correctly, and prevent autonomous use of write-capable MCP tools.

 Repository MCP configuration draft

```
{
  "mcpServers": {
    "github-mcp-server": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "tools": ["*"],
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    },
    "sentry": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@sentry/mcp-server"],
      "tools": ["*"],
      "env": {
        "SENTRY_AUTH_TOKEN": "${SENTRY_AUTH_TOKEN}"
      }
    }
  }
}
```

- A** Use built-in read-only GitHub MCP, allowlist Sentry read tools, and reference a `COPILOT_MCP_secret`. ✓
- B** Keep the GitHub entry, document the broad tools, and store `SENTRY_AUTH_TOKEN` as an Actions secret.
- C** Change only the GitHub URL to `/readonly`, keep tools `["*"]`, and validate after the first run.
- D** Move the MCP setup into `copilot-setup-steps.yml` and grant contents: read to the setup job.

 Why the correct answer is right

Copilot cloud agent has a built-in GitHub MCP server with read-only access to the current repository, so a broader custom GitHub MCP entry is unnecessary for this scenario. MCP configurations should allowlist specific tools, especially read-only tools, because the agent can use

MCP tools autonomously. Secret references in MCP configuration must come from Copilot environment variables or secrets with names prefixed by `COPILOT_MCP_`.

✗ Why the other options are wrong

- **B:** Actions secrets are not the right substitution source for this MCP configuration, and documenting broad access does not reduce autonomous tool exposure.
- **C:** The `/readonly` endpoint is safer than the broad endpoint, but leaving tools as `["*"]` and keeping invalid secret substitution is incomplete.
- **D:** `copilot-setup-steps.yml` customizes the agent development environment; it is not where repository MCP server authorization and tool allowlists are configured.

💡 When an artifact mixes environment setup, MCP configuration, and secrets, separate the control planes: MCP tool allowlists belong in MCP config, and MCP credentials must use Copilot environment secret naming.

mcp-configuration

copilot-environment-secrets

github-mcp-server

safe-tool-execution

Q4**Domain E: Multi-agent orchestration and incident response****code or config artifact****hard**

A platform team is building an incident-review dashboard for a Copilot SDK workflow that delegates research, implementation, and validation work to custom sub-agents. After a failed release candidate, reviewers cannot determine which sub-agent invocation introduced the bad change because the same agent name appears in multiple places. What should the team change?

The workflow must preserve enough operational evidence to reconstruct each sub-agent execution after the fact without disabling automatic delegation.

 Current event subscriber

```
session.on((event) => {
  if (event.type === "subagent.selected") {
    audit.write({
      agent: event.data.agentName,
      tools: event.data.tools,
      status: "selected",
      at: event.timestamp
    });
  }

  if (event.type === "subagent.deselected") {
    audit.write({
      status: "returned-to-parent",
      at: event.timestamp
    });
  }
});
```

- A** Track started, completed, and failed events keyed by toolCallId, including status, error, and timestamps. ✓
- B** Set infer:false on worker agents and require operators to invoke each agent explicitly during incidents.
- C** Preselect the implementation agent at session creation so all emitted events share one parent context.
- D** Group events by agentName and overwrite the previous status whenever the same agent runs again.

✓ **Why the correct answer is right**

GitHub documents that sub-agent lifecycle events include started, completed, and failed events, and that toolCallId fields let you reconstruct the execution tree. Capturing those events with status, errors, and timestamps gives reviewers the audit trail needed for post-hoc failure analysis.

✗ **Why the other options are wrong**

- **B:** Disabling inference changes delegation behavior but does not create a reliable execution tree for incident review.
- **D:** Grouping only by agentName collapses separate invocations of the same agent and can hide the failed execution.
- **C:** Preselecting one agent narrows the session start context but does not capture lifecycle status across sub-agents.

💡 For GH-600 observability questions, prefer durable, correlatable artifacts over changes that merely constrain routing. Lifecycle events plus correlation identifiers are stronger than agent names alone.

multi-agent-observability

subagent-events

post-hoc-analysis

incident-response

Q5**Domain A: Agent architecture and SDLC integration**

single choice

hard

A platform team is piloting Copilot coding agent for dependency remediation in a regulated repository. Leadership wants the agent to reduce developer effort, but the security team requires traceable work, independent human review, and CI evidence before any change reaches the default branch. Which SDLC design best satisfies these requirements?

The repository already uses GitHub pull requests, CODEOWNERS, required checks, and protected branches. The team wants to avoid adding a separate approval system outside GitHub.

- A** Keep Copilot work in pull requests, require status checks and CODEOWNERS review, then have a maintainer approve and merge. ✓
- B** Run Copilot in a staging fork, then have maintainers copy the final patch into the protected repository.
- C** Enable auto-merge for Copilot pull requests after dependency tests pass, then review agent session logs during weekly audits.
- D** Require Copilot to post an issue comment plan, then allow deployment once the package manager test command succeeds.

✓ **Why the correct answer is right**

Using pull requests as the control point preserves GitHub as the system of record, allows required status checks and CODEOWNERS review to gate the merge, and aligns with Copilot coding agent behavior: Copilot proposes changes for review and cannot approve or merge its own pull request.

✗ **Why the other options are wrong**

- **C:** This shifts control to after-the-fact auditing and allows merge based mainly on tests, rather than requiring independent review before integration.
- **D:** A plan comment is useful, but it does not provide sufficient PR-based governance, review enforcement, or merge protection by itself.
- **B:** A staging fork may reduce direct access, but copying patches weakens attribution, review traceability, and normal GitHub PR controls.

💡 For GH-600 SDLC questions, prefer designs that use GitHub-native checkpoints: PRs, required checks, CODEOWNERS, protected branches, and human approval before merge.

sdlc-integration

human-approval-gates

pr-governance

agent-control

Q6**Domain B: MCP/tool use and environment permissions****single choice****hard**

A platform team is creating a custom Copilot cloud agent to investigate flaky test failures. The agent must read source files, search the repository, and retrieve issue metadata from an MCP server, but it must not edit files or call write-capable MCP tools. Which configuration approach should the team use?

The repository already has an MCP server configured with both read and write tools. The team wants the agent profile itself to enforce least-privilege access.

- A Move write-capable MCP tools to a separate server but keep both servers configured.
- B Set tools to ["*"] and rely on system instructions not to modify files.
- C Omit tools so the runtime can choose the smallest set needed for each task.
- D List only read and search aliases plus required server-prefixed read-only MCP tools. ✓**

✓ **Why the correct answer is right**

A specific tools list filters the built-in and MCP tools made available to the custom agent. Using read/search aliases and server-prefixed read-only MCP tool names enforces the agent's required capability boundary instead of depending on instructions alone.

✗ **Why the other options are wrong**

- **C:** Omitting the tools property enables all available tools, which can expose write-capable MCP tools already configured for the repository.
- **B:** Using ["*"] enables all available tools, so the agent profile does not enforce the required read-only boundary.
- **A:** Separating servers can help organization, but keeping both servers configured does not restrict which tools this agent may call.

💡 For GH-600 tool-permission questions, prefer an explicit allowlist at the narrowest practical scope. Prompts, comments, or server reorganization do not replace tool filtering.

agent-tool-permissions

mcp-tool-scoping

least-privilege

Q7**Domain D: Evaluation, telemetry, error analysis, and tuning****code or config artifact****hard**

A backend team uses a custom implementation-planner agent before allowing Copilot cloud agent to modify service code. After three sprints, reviewers report that the final PRs often solve a related problem but do not match the agent's original plan. You review the session log excerpt and agent profile. Which tuning action best addresses the root cause?

The team wants to improve plan adherence without blocking valid agent work or replacing the agent with a manual review process.

 Session log excerpt and current custom agent profile

```
SESSION: copilot/plan-order-retry-429
TASK: Add retry handling for order submission timeouts.

[plan]
- Update OrderClient to retry HTTP 429 and timeout responses.
- Add tests for timeout retry behavior.
- Keep checkout API response shape unchanged.

[tool: search]
query: "OrderClient retry timeout"
result: src/payments/PaymentClient.ts has retry helper.

[tool: edit]
files changed:
- src/api/CheckoutController.ts
- src/ui/messages.ts
- package-lock.json

[validation]
npm test -- --filter checkout => passed

[agent note]
Implemented a broader checkout retry UX because the existing retry helper was in
payments.

--- .github/agents/implementation-planner.agent.md ---
---
name: implementation-planner
description: Creates implementation plans for backend changes
tools: ["read", "search", "edit"]
---
Create a concise implementation plan. Prefer existing patterns. If tests pass, pr
epare the result for review.
```

- A** Increase the model tier and token budget, then monitor whether plan drift decreases over the next sprint.

B Add required CodeQL and secret scanning checks so unrelated PR changes are caught during security validation.

C Revise the agent instructions to require success criteria, constraints, risks, and plan-to-diff validation before review. ✓

D Remove edit access from the planner agent and require developers to manually translate every generated plan.

✓ **Why the correct answer is right**

The log shows a prompt/instruction issue and plan-adherence failure: the agent produced a related implementation that changed files outside the stated plan and lacked explicit success criteria or constraints. The documented implementation-planner pattern includes success criteria, technical approach, phased tasks, assumptions, constraints, and risks; session logs are the right evidence for comparing the plan, tool use, validation, and final changes.

✗ **Why the other options are wrong**

- **B:** Security scanning is useful for vulnerabilities and secrets, but it does not directly address plan drift or missing task constraints.
- **A:** A larger model or token budget may improve some outputs, but it does not encode the missing evaluation criteria or constraints.
- **D:** Removing edit access avoids the symptom by disabling useful agent work, rather than tuning the instructions that caused the drift.

💡 For GH-600 tuning questions, identify the failure class first. If the plan is reasonable but execution drifts because constraints are vague, prefer targeted instruction and evaluation-signal changes over broader monitoring or capability removal.

failure-analysis

evaluation-signals

agent-instruction-tuning

plan-adherence

 Case study — Case Study: HelioLedger Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

HelioLedger is a financial software company that maintains a GitHub Enterprise Cloud organization for payment reconciliation, ledger reporting, and customer support tooling. The company is piloting GitHub Copilot cloud agent and custom agents to reduce the backlog of low-risk maintenance issues while keeping regulated repositories auditable.

Existing Environment

The organization uses Copilot Business. The pilot repositories include ``ledger-api``, ``support-portal``, and ``reconciliation-workers``. Issues are created by product owners and must include affected service, expected behavior, acceptance criteria, test expectations, and known risky files. The platform team has created a ``github-private`` repository from GitHub's custom agents template, but most teams have not yet standardized how agent profiles should separate planning from implementation.

Business Requirements

HelioLedger wants agents to handle routine bug fixes, dependency updates, and internal documentation corrections. Agent work must remain visible in GitHub issues, pull requests, comments, commits, checks, and logs. Reviewers must be able to understand why an agent made a change without opening external systems.

Technical Requirements

All agent-created implementation work must use pull requests. The desired workflow is: define task inputs and success criteria, produce an inspectable plan, wait for a human checkpoint when code or workflow changes are involved, make changes on an agent branch, run applicable validation, and request human review. The organization wants to use custom agents for specialized roles such as ``support-fix-planner`` and ``safe-implementation-agent``.

Security and Compliance Requirements

The compliance team requires independent review before merge, required status checks, CODEOWNERS review for ``github/``, ``github/workflows/``, ``github/copilot-instructions.md``, ``copilot-setup-steps.yml``, and agent profile changes. Agent-created ``copilot/`` branches must not bypass repository rules. Deployments to protected environments must require manual approval. Audit evidence must include the initiating user, agent-authored commits, session logs, and the final pull request review outcome.

HelioLedger is defining the standard workflow for routine agent fixes in ``ledger-api``. Order the steps to satisfy the required plan, act, and evaluate lifecycle while preserving the required human checkpoint.

The workflow must define clear task inputs, prevent code changes until the plan is approved, and keep review evidence inside GitHub.

Correct order:

1. Extract the issue inputs, affected paths, acceptance criteria, test expectations, and known risky files.
2. Publish a structured implementation plan and wait for the required reviewer checkpoint in the issue or pull request.
3. After approval, implement the change on the agent branch and run the applicable checks.
4. Create or update the draft pull request with validation results, session evidence, and a human review request.

✓ **Why the correct answer is right**

The workflow begins by turning the issue into structured inputs and success criteria. The agent then publishes a plan and waits for approval. After that checkpoint, it can act on an agent branch and run validation. The final step is evaluation through pull request review, checks, and traceable evidence.

✗ **Why the other options are wrong**

- **A:** Creating the review package is an evaluation activity and belongs after implementation and validation, not before planning.
- **B:** A plan is valuable only after the task inputs and success criteria have been extracted from the issue.
- **D:** Implementation must occur after the structured plan has been reviewed under HelioLedger's required checkpoint.

💡 Sequence items often test lifecycle boundaries. Identify the earliest input-definition step, then place human approval before any irreversible or code-changing action.

plan-act-evaluate

sdic-integration

agent-workflow-sequencing

 Case study — Case Study: HelioLedger Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

HelioLedger is a financial software company that maintains a GitHub Enterprise Cloud organization for payment reconciliation, ledger reporting, and customer support tooling. The company is piloting GitHub Copilot cloud agent and custom agents to reduce the backlog of low-risk maintenance issues while keeping regulated repositories auditable.

Existing Environment

The organization uses Copilot Business. The pilot repositories include ``ledger-api``, ``support-portal``, and ``reconciliation-workers``. Issues are created by product owners and must include affected service, expected behavior, acceptance criteria, test expectations, and known risky files. The platform team has created a ``github-private`` repository from GitHub's custom agents template, but most teams have not yet standardized how agent profiles should separate planning from implementation.

Business Requirements

HelioLedger wants agents to handle routine bug fixes, dependency updates, and internal documentation corrections. Agent work must remain visible in GitHub issues, pull requests, comments, commits, checks, and logs. Reviewers must be able to understand why an agent made a change without opening external systems.

Technical Requirements

All agent-created implementation work must use pull requests. The desired workflow is: define task inputs and success criteria, produce an inspectable plan, wait for a human checkpoint when code or workflow changes are involved, make changes on an agent branch, run applicable validation, and request human review. The organization wants to use custom agents for specialized roles such as ``support-fix-planner`` and ``safe-implementation-agent``.

Security and Compliance Requirements

The compliance team requires independent review before merge, required status checks, CODEOWNERS review for ``github/``, ``github/workflows/``, ``github/copilot-instructions.md``, ``copilot-setup-steps.yml``, and agent profile changes. Agent-created ``copilot/`` branches must not bypass repository rules. Deployments to protected environments must require manual approval. Audit evidence must include the initiating user, agent-authored commits, session logs, and the final pull request review outcome.

HelioLedger's ``support-fix-planner`` agent will triage customer-impacting issues in ``support-portal``, but auditors require reviewers to see the agent's intended approach before any code changes are made. Which design best satisfies the case study requirements?

The team wants to keep delivery fast, but the workflow must preserve an inspectable planning artifact and prevent implementation before a human checkpoint.

- A Keep the plan in the session log and allow the agent to mark the pull request ready.
- B Have a project manager write a separate planning document, then let the agent implement from that document.
- C Let the agent implement the fix first, then require CI and reviewers to catch problems in the pull request.
- D Require a structured plan comment with inputs, tests, and success criteria, then wait for approval before changes. ✓

✓ **Why the correct answer is right**

The correct design separates planning from action, produces a structured artifact in GitHub, defines inputs and success criteria, and adds a human checkpoint before implementation. This aligns with the GH-600 objective to separate planning, reasoning, and execution and with Copilot cloud agent's GitHub-centered review workflow.

✗ **Why the other options are wrong**

- **C:** This relies on evaluation after implementation and does not prevent the agent from acting before the plan is reviewed.
- **B:** This creates a planning artifact, but it moves the inspectable agent plan outside the agent workflow and weakens traceability.
- **A:** Session logs help auditing, but the plan is not placed in the normal review surface, and Copilot cannot mark its own pull requests ready.

💡 For GH-600 planning questions, prefer answers that create an inspectable GitHub artifact before action and insert human approval at the risk boundary.

planning-execution-boundary

human-checkpoint

structured-agent-output

Q10**Domain A: Prepare agent architecture and SDLC processes**

single choice

hard

 Case study — Case Study: HelioLedger Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

HelioLedger is a financial software company that maintains a GitHub Enterprise Cloud organization for payment reconciliation, ledger reporting, and customer support tooling. The company is piloting GitHub Copilot cloud agent and custom agents to reduce the backlog of low-risk maintenance issues while keeping regulated repositories auditable.

Existing Environment

The organization uses Copilot Business. The pilot repositories include ``ledger-api``, ``support-portal``, and ``reconciliation-workers``. Issues are created by product owners and must include affected service, expected behavior, acceptance criteria, test expectations, and known risky files. The platform team has created a ``github-private`` repository from GitHub's custom agents template, but most teams have not yet standardized how agent profiles should separate planning from implementation.

Business Requirements

HelioLedger wants agents to handle routine bug fixes, dependency updates, and internal documentation corrections. Agent work must remain visible in GitHub issues, pull requests, comments, commits, checks, and logs. Reviewers must be able to understand why an agent made a change without opening external systems.

Technical Requirements

All agent-created implementation work must use pull requests. The desired workflow is: define task inputs and success criteria, produce an inspectable plan, wait for a human checkpoint when code or workflow changes are involved, make changes on an agent branch, run applicable validation, and request human review. The organization wants to use custom agents for specialized roles such as ``support-fix-planner`` and ``safe-implementation-agent``.

Security and Compliance Requirements

The compliance team requires independent review before merge, required status checks, CODEOWNERS review for ``github/``, ``github/workflows/``, ``github/copilot-instructions.md``, ``copilot-setup-steps.yml``, and agent profile changes. Agent-created ``copilot/`` branches must not bypass repository rules. Deployments to protected environments must require manual approval. Audit evidence must include the initiating user, agent-authored commits, session logs, and the final pull request review outcome.

HelioLedger's compliance team is concerned that agent-created changes could modify workflows, setup steps, or custom agent definitions without the right reviewers. Which control set best satisfies the case study requirements without blocking all agent work?

The organization wants agents to continue creating routine fixes, but sensitive configuration and deployment paths must require stronger review and validation.

- A Add CODEOWNERS entries for sensitive files, but allow agent pull requests to merge when CI succeeds.
- B Enable the agent firewall and review audit logs weekly for changes to workflows, setup steps, and agent profiles.
- C Disable automatic workflow runs for agent pull requests and rely on reviewers to inspect session logs.
- D Use rulesets for `copilot/` branches requiring PRs, checks, CODEOWNERS review, and protected environment approval. ✓

✓ **Why the correct answer is right**

Rulesets or branch protection targeted at agent-created branches enforce pull requests, required checks, and human review. CODEOWNERS protects sensitive GitHub configuration files, while protected environments add the manual deployment gate required by the case study.

✗ **Why the other options are wrong**

- **B:** Firewall and logs help reduce exfiltration risk and support auditability, but they do not enforce merge controls for sensitive files.
- **A:** CODEOWNERS identifies reviewers, but without required review and merge rules it does not fully enforce the compliance requirement.
- **C:** Manual inspection is useful, but session logs alone do not enforce required checks, CODEOWNERS review, or deployment approval.

💡 When a scenario asks for governance without stopping delivery, select preventive GitHub controls: PRs, rulesets or branch protection, CODEOWNERS, checks, and environment gates.

pr-governance

agent-control-plane

branch-protection-codeowners

Quick Answer Key

#	Type	Correct Answer
Q1	single choice	B
Q2	single choice	C
Q3	code or config artifact	A
Q4	code or config artifact	A
Q5	single choice	A
Q6	single choice	D
Q7	code or config artifact	C
Q8	sequence order	C → B → D → A
Q9	single choice	D
Q10	single choice	D